SUDOS: Smart Urban Delivery Optimization System

Algorithmic Approaches to Urban Last-Mile Delivery

Pranad Bajoria

24BAI0352

Yug Shah

24BCE0286

Sreerag Aneesh Nair

24BCE0081

Dhruv Sharma

24BCE0085

Course: Design and Analysis of Algorithms

Urban Delivery Challenge

Modern delivery platforms handle thousands of orders daily. Even small improvements scale massively in production systems.

Assignment Problem

Which agent handles which deliveries?

Routing Problem

In what order should locations be visited?

Optimization Goal

Minimize total travel distance across all agents

A **5% distance reduction** = significant cost savings + faster deliveries. Related to VRP and TSP — classic NP-hard problems.

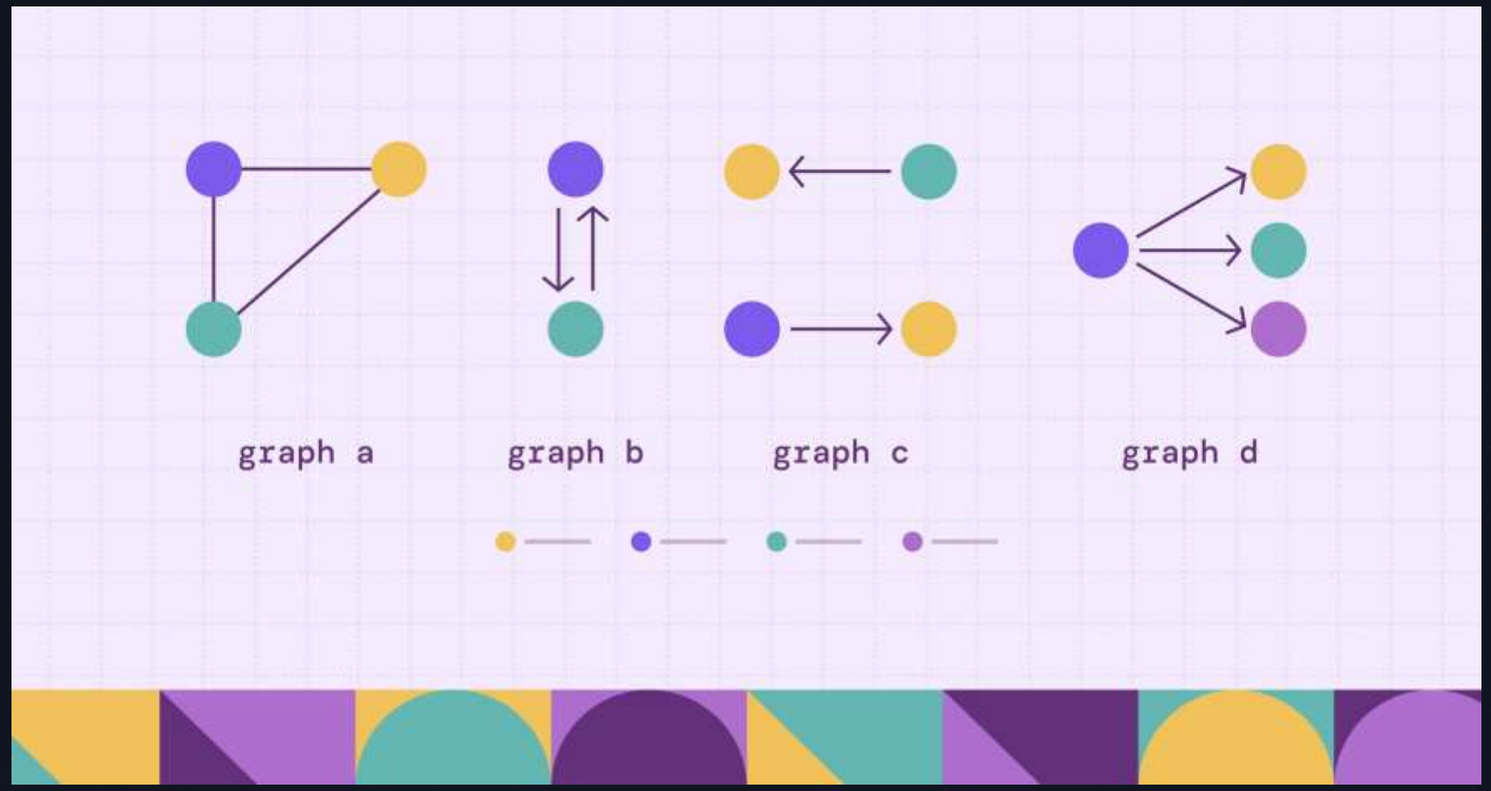
SUDOS Pipeline — Five Stage Approach

Stage 4: Three Parallel Approaches

Greedy Nearest Neighbor · Bitmask DP-TSP · Nearest Insertion

Stage 5: Metrics

Total distance & runtime compared across all methods



Foundation Layer: Dijkstra's Shortest Path

Run once per location to build a complete all-pairs shortest-path matrix, enabling **O(1) distance lookups** during routing.

Precomputation separates graph structure from routing logic — a key architectural insight.

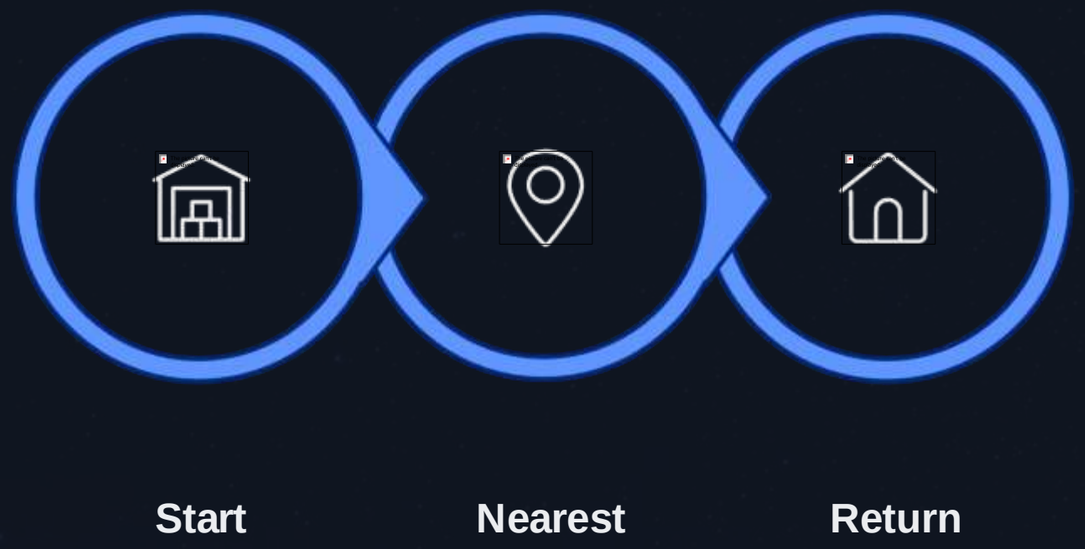
Per Source

$O(V^2)$ array-based

All Pairs

$O(V^3)$ total

Fast & Practical: The Greedy Approach



Time Complexity

$O(n^2)$ per agent

Optimality

Not guaranteed — up to 20–25% suboptimal

Best For

100+ deliveries, real-time re-routing, quick baselines

Analogy: How a human naturally plans a delivery route.

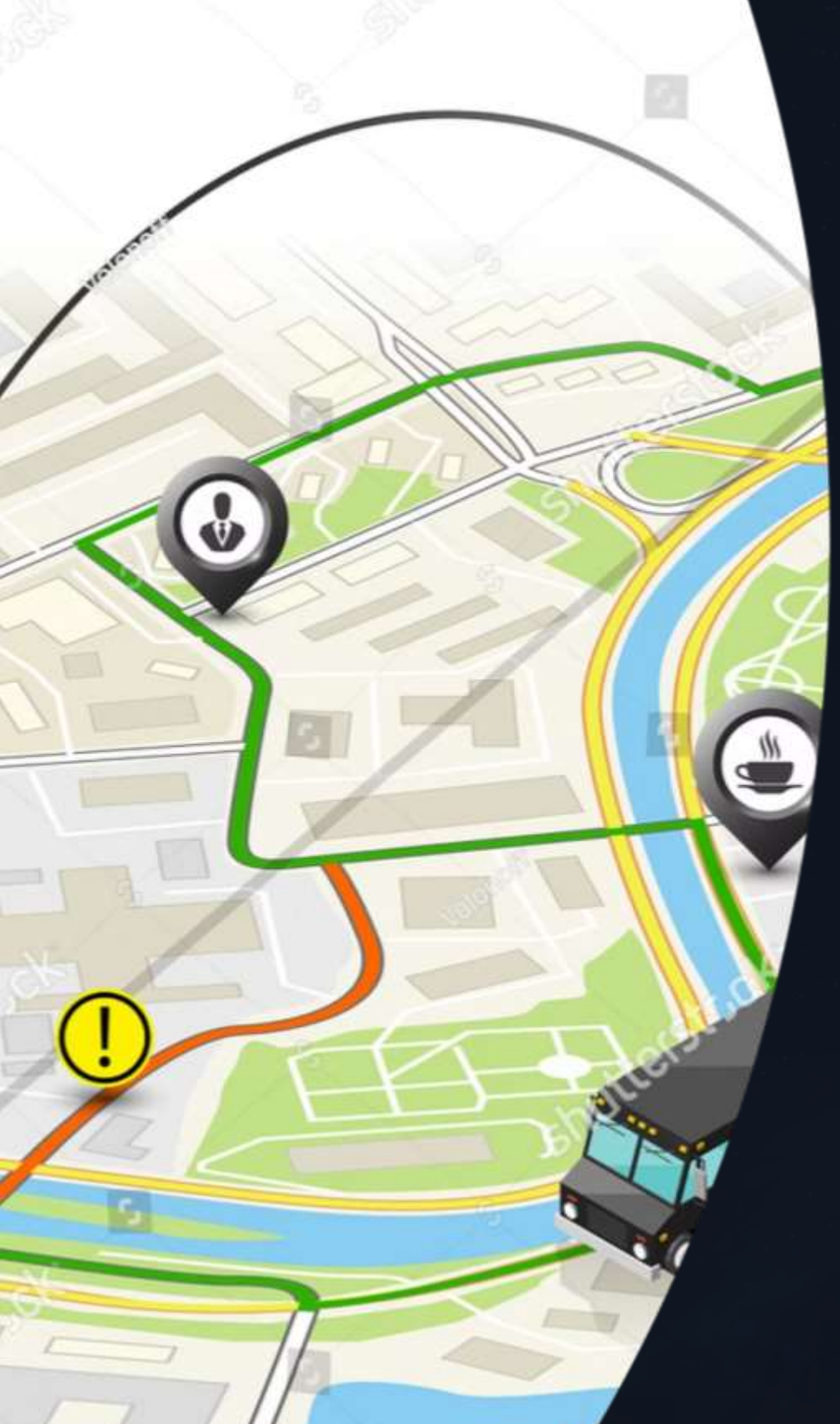
The Exact Solution: Bitmask DP-TSP

The **Held-Karp Algorithm** uses bitmask states — `dp[mask][i]` = min cost to visit locations in *mask*, ending at node *i* — with memoization over all orderings.

Time $O(2^n \times n^2)$	**Space** $O(2^n \times n)$
Optimality Guaranteed optimal	**Practical Limit** $n \leq 15$ nodes

~1ms	~2s	∞
n = 10	n = 15	n = 20 (Infeasible)

Purpose: Benchmark for comparing heuristic methods.



The Sweet Spot: Nearest Insertion Heuristic

How It Works

Start with Depot → nearest node → Depot. Repeatedly insert the unvisited node that adds minimum extra distance.

Performance

$O(n^2)$ time · ≤ 2× optimal guarantee · Typically 1.1–1.4× optimal in practice

Best Use Case

Production systems requiring quality + speed. Handles 1000+ nodes efficiently.

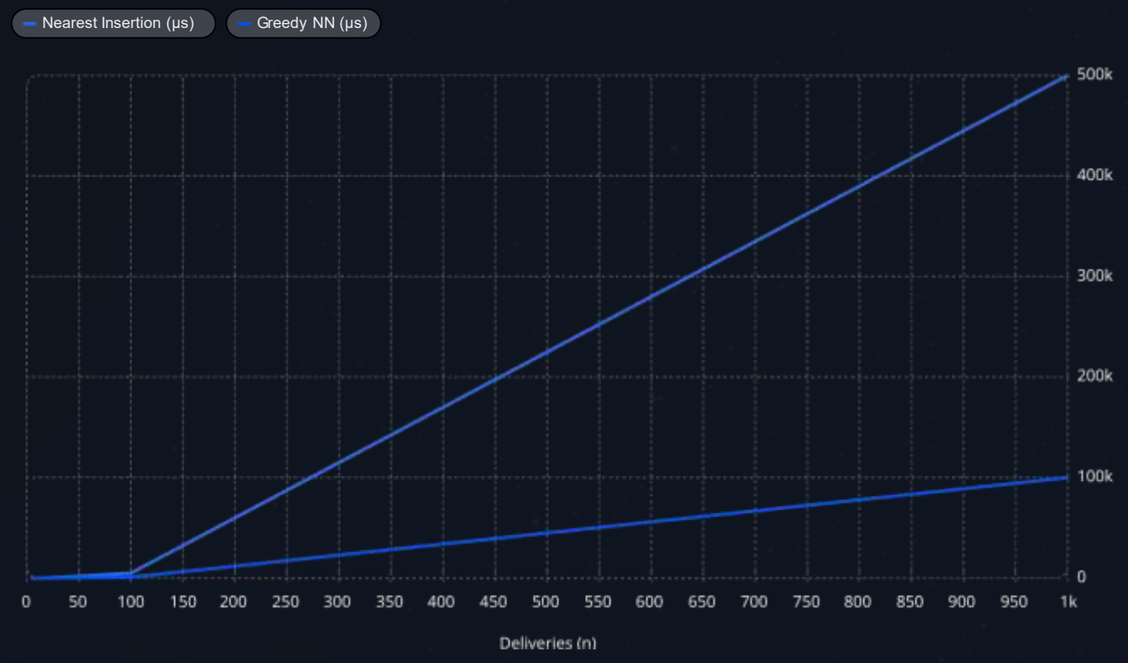
Performance Comparison — 8 Node City Graph

Test: 8 locations · 2 agents (4 and 3 deliveries) · Round-robin assignment

Algorithm	Agent 1 Cost	Agent 2 Cost	Total Cost	Runtime
Greedy NN	83	75	158	<1 μs
DP-TSP (Optimal)	83	75	158	5–50 μs
Nearest Insertion	83	75	158	1–5 μs

Optimality Gap: 0% — All three methods found the optimal route on this instance. On smaller problems, even greedy can be optimal.

The Exponential Wall — Why DP Doesn't Scale



DP-TSP

Perfect for n ≤ 12. Infeasible beyond n = 20 (memory & time).

Heuristics

Only viable option at real-world scale (50–500 stops per agent).

Production Reality

Memory becomes the limiting factor before runtime does.

Key Learnings & Next Steps

✓ No Single Winner

Problem size dictates algorithm choice.

✓ Precompute First

Dijkstra simplifies all downstream routing.

✓ Best Balance

Nearest Insertion wins on quality + speed.

Future Enhancements

Smarter Assignment

Min-cost flow (Ford-Fulkerson) for spatial clustering instead of round-robin.

Time Windows

Add delivery deadlines (Time-Windowed VRP) with feasibility checks.

Metaheuristics

Simulated Annealing & Genetic Algorithms for 500+ deliveries per agent.

Dynamic Re-routing

Sliding window re-optimization adapting to real-time traffic and new orders.